Programação para Internet: MVP Funcional e Integração com Gestão

Projeto HermesPDV — Aula 6

O MVP do HermesPDV está funcionando: aplicação web com front-end e back-end integrados ao banco PostgreSQL da Aula 5, os requisitos de prioridade alta da Aula 2, as regras de negócio em código e dois pontos de integração com a gestão (dashboard gerencial e API própria).

Entregáveis

Entregável	Onde está	Situação
Link do repositório com código completo e README	Seção 8 (link) e README.md na raiz	Pendente: colar o link
Aplicação publicada ou vídeo de 5 a 8 minutos	HermesPDV_Aula6_demonstracao_MVP.mp4 (7 min 21 s)	Pronto
Relatório de sprint	Seção 2	Pronto
Print do quadro de tarefas	Seção 3	Pendente: colar o print
Documento da integração escolhida	Seção 4	Pronto
Tabela de rastreabilidade	Seção 5	Pronto

1. O MVP entregue

1.1 Escopo mínimo da aula × o que foi feito

Item	Exigência	O que o HermesPDV faz	Situação
Autenticação	Login com usuário e senha, senha com hash, controle de sessão e ao menos dois perfis	Login com bcrypt (pgcrypto); sessão em cookie HttpOnly com token JWT de 8 h; logout; bloqueio por 1 minuto após 5 erros; três perfis (Administrador, Gerente, Atendente) checados no servidor	Implementado
CRUDs	No mínimo 3 cadastros completos com validação	Produtos (+ reposição de estoque), clientes (CPF/CNPJ com dígito verificador) e usuários (senha forte), com validação na tela e no servidor	Implementado
Transação principal	Fluxo central do domínio	Registrar venda: carrinho, cliente, desconto, pagamento dividido com troco, gravação em uma transação, baixa de estoque, alerta e comprovante; cancelamento com motivo	Implementado
Regras de negócio	Pelo menos 3 regras em código	Nove regras (RN01 a RN09) aplicadas no servidor; várias também protegidas por restrições do banco	Implementado
Consultas e filtros	Busca e filtro por período ou status	Vendas por período, status, número, cliente e produto; produtos por nome/código, estoque baixo e inativos; clientes e usuários por nome	Implementado
Relatório gerencial	Totalizadores, exportável em PDF ou CSV	Relatório de vendas com totalizadores, por dia, por forma de pagamento e mais vendidos; exporta CSV (Excel) e imprime/salva PDF	Implementado
Interface	Layout responsivo, mensagens claras	Funciona em celular e computador; avisos de sucesso, alerta e erro em português; nenhuma mensagem técnica do banco chega ao usuário	Implementado
Integração com gestão	Ao menos uma opção, documentada	Módulo gerencial (dashboard com os indicadores da Aula 3) e API própria de vendas em JSON (seção 4)	Implementado

1.2 Perfis de acesso (RN04)

A permissão é verificada no servidor em cada rota da API; o menu de cada perfil só mostra o que ele pode usar. Tentar abrir uma tela proibida pelo endereço leva de volta ao painel com aviso.

Função	Atendente	Gerente	Administrador
Painel	Atalhos e alertas	Dashboard gerencial	Dashboard gerencial
Nova venda (PDV) e cancelamento	Sim	—	—
Vendas e comprovantes	Consulta	Consulta	Consulta
Produtos	Consulta de estoque	Consulta	Cadastro completo e reposição
Clientes	Consulta	Consulta	Cadastro completo
Usuários	—	—	Cadastro completo
Relatórios, exportação e API de integração	—	Sim	Sim

1.3 Como executar e usuários de teste

```
cd banco
psql -U postgres -f script_criacao.sql
psql -U postgres -d hermespdv -f script_carga.sql
cd ../backend
npm install
cp .env.example .env      # no Windows: copy .env.example .env — depois coloque a senha do PostgreSQL
npm start                 # http://localhost:3000
```

Usuários de teste (senha Hermes@2026): helena.adm (Administrador), rafael.gerente (Gerente) e bianca.caixa (Atendente). Os dados de teste são fictícios.

Mudanças no banco desde a Aula 5: extensão pgcrypto (hash bcrypt pelo próprio PostgreSQL), fuso horário de Brasília e a coluna venda.iniciada_em, necessária para o indicador de tempo de atendimento. Quem já tem o banco da Aula 5 roda banco/migracao_aula6.sql.

2. Relatório de sprint

Sprint da Aula 6: de 04/11 a 15/11/2026. Objetivo: entregar o MVP navegável, com os fluxos dos três perfis completos e conectados ao banco.

2.1 Planejado × entregue

Item planejado	Resultado	Observação
Autenticação com perfis (RF04, RN04, RNF02)	Entregue	Login, sessão, logout, bloqueio de tentativas e permissões por rota.
Cadastros de produtos, clientes e usuários (RF01–RF03)	Entregue	Com validações, mensagens e as regras RN05 e RN06.
Registrar venda com pagamento dividido (RF05–RF09)	Entregue	Inclui troco (RN09), alerta de estoque (RN07) e comprovante (RF13).
Consulta e cancelamento de vendas (RF11, RF12)	Entregue	Filtros por período, status, número, cliente e produto; motivo obrigatório (RN08).
Relatório gerencial (RF14–RF16) exportável	Entregue	CSV para Excel e impressão/PDF.
Dashboard com os 4 indicadores da Aula 3	Parcial	3 de 4 indicadores. “Erros de caixa por dia” depende de um fechamento de caixa (contagem da gaveta), que não existe nos requisitos.
API de integração	Entregue	GET /api/integracao/vendas, documentada e consumida com curl.
Testes automatizados	Entregue	15 testes de API com node:test, sem dependências extras.
Vídeo de demonstração (5 a 8 min)	Entregue	7 min 21 s, com legendas, mostrando os fluxos dos três perfis e o celular.
Publicação na nuvem	Pendente	Depende de conta da equipe no serviço de hospedagem; passo a passo pronto no README.
Commits de todos os integrantes, branches e PRs	Pendente	Responsabilidade da equipe; plano de divisão na seção 7.

2.2 O que ficou pendente e por quê

Pendência	Por que ficou de fora	Próximo passo
Indicador “erros de caixa por dia” (Aula 3)	Exige registrar a contagem física do dinheiro no fim do dia (fechamento de caixa), função que não está nos requisitos da Aula 2.	Incluir no backlog uma história de fechamento de caixa e decidir com o professor se entra na entrega final.
Publicação online	É preciso criar conta no serviço e guardar a senha do banco; isso cabe à equipe.	Publicar seguindo o README (Render ou Railway) ou entregar o vídeo, como a aula permite.
Cadastro de categorias pela tela	As 10 categorias vêm do script de carga; o RF01 pede só a categoria no produto. Prioridade baixa.	Tela simples de categorias, reaproveitando o padrão de clientes.
Recuperação de senha	Precisaria de envio de e-mail; o administrador já redefine a senha na tela de usuários.	Fora do escopo do MVP.
Integrações TEF e NF-e	Fora do escopo desde a Aula 1.	—

2.3 Decisões e aprendizados da sprint

- O hash e a conferência da senha ficaram no PostgreSQL (pgcrypto), o que eliminou uma dependência e manteve os hashes bcrypt da carga de teste.
- Os valores em dinheiro são calculados em centavos inteiros no servidor, para não haver erro de arredondamento entre itens, desconto e pagamentos.
- O preço da venda vem sempre do banco, nunca da tela: quem altera o HTML não muda o preço cobrado.
- Venda e cancelamento travam as linhas dos produtos (SELECT … FOR UPDATE) dentro da transação, o que impede que duas vendas simultâneas usem o mesmo saldo.
- As regras sugeridas na Aula 4 (RN08, motivo do cancelamento, e RN09, troco) foram adotadas e implementadas.

3. Quadro de tarefas

Print do quadro (GitHub Projects, Trello ou similar) com os cartões concluídos nesta sprint.

Cole aqui o print do quadro de tarefas.

Sugestão de cartões, se o quadro ainda não estiver atualizado:

- Login, sessão e perfis
- CRUD de produtos com reposição de estoque
- CRUD de clientes com validação de CPF/CNPJ
- CRUD de usuários com senha forte
- PDV: carrinho, cliente e desconto
- Pagamento dividido, troco e finalização
- Comprovante
- Consulta e cancelamento de vendas
- Relatório com CSV e PDF
- Dashboard com os indicadores da Aula 3
- API de integração
- Testes automáticos
- Vídeo de demonstração

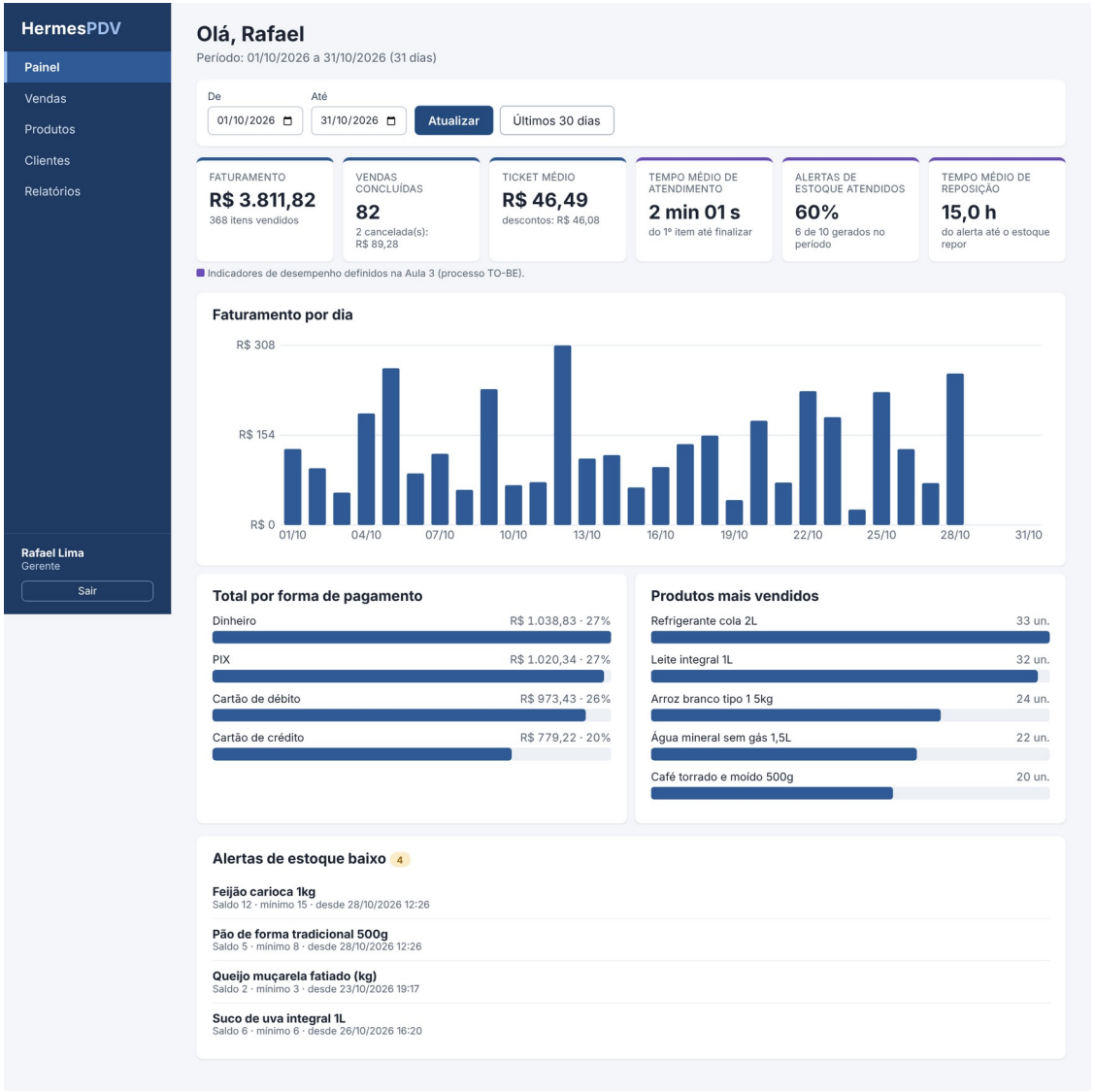
4. Integração com a gestão

Foram implementadas duas das opções da aula: o módulo gerencial (dashboard com os indicadores definidos na Aula 3) como integração principal, e uma API própria que entrega as vendas a outros sistemas, como um ERP ou uma planilha.

4.1 Módulo gerencial: dashboard

Disponível para Gerente e Administrador na tela Painel. A tela chama GET /api/dashboard?inicio=&fim= e o servidor calcula tudo com consultas SQL de agregação. Sem período informado, usa os últimos 30 dias até a venda mais recente. Os gráficos são desenhados em SVG pelo próprio JavaScript, sem bibliotecas externas.

Indicador	Fórmula	Fonte de dados	Outubro/2026	Situação
Tempo médio de atendimento no caixa	média de (data_hora − iniciada_em) das vendas concluídas	Coluna venda.iniciada_em, gravada quando o 1º item entra no carrinho	2 min 01 s	Implementado
Percentual de rupturas evitadas	alertas resolvidos ÷ alertas gerados no período × 100	Tabela alerta_estoque (resolvido, resolvido_em)	60% (6 de 10)	Implementado
Tempo médio de reposição de estoque	média de (resolvido_em − data_hora) dos alertas resolvidos	Tabela alerta_estoque; a reposição pela tela encerra o alerta	15,0 h	Implementado
Número de erros de caixa por dia	|valor contado na gaveta − valor registrado|, por dia	Depende de fechamento de caixa (não existe ainda)	—	Não implementado
Faturamento, vendas, ticket médio, cancelamentos	soma, contagem e média das vendas do período	Tabelas venda e pagamento	R$ 3.811,82 · 82 · R$ 46,49 · 2	Implementado



Dashboard do gerente em outubro/2026. Os cartões roxos são os indicadores da Aula 3.

4.2 API própria de integração

Item	Definição
Endpoint	GET /api/integracao/vendas?inicio=AAAA-MM-DD&fim=AAAA-MM-DD
Autenticação	Sessão de um usuário Gerente ou Administrador: cookie do login (POST /api/auth/login) ou cabeçalho Authorization: Bearer <token>
Resposta	JSON, UTF-8. Somente vendas concluídas; canceladas ficam de fora.
Erros	400 (data inválida ou período acima de 1 ano), 401 (sem sessão), 403 (perfil sem permissão), mensagens em português no campo "erro"

Layout do arquivo (versão 1.0):

Campo	Tipo	Descrição
`sistema, versao_layout`	texto	Identificação do emissor e versão do layout (1.0).
`gerado_em`	data/hora ISO 8601 (UTC)	Momento em que o arquivo foi gerado.
`periodo.inicio, periodo.fim`	data AAAA-MM-DD	Período consultado.
`quantidade, total`	inteiro, decimal	Número de vendas e soma dos totais (conferência).
`vendas[].numero`	inteiro	Número da venda no HermesPDV (chave para evitar importação duplicada).
`vendas[].data_hora`	data/hora local (Brasília)	Momento da finalização da venda.
`vendas[].cliente`	objeto ou null	nome e documento (CPF/CNPJ só com números).
`vendas[].atendente`	texto	Login de quem registrou a venda.
`vendas[].subtotal, desconto, total`	decimal (2 casas)	Valores da venda; total = subtotal − desconto.
`vendas[].itens[]`	lista	codigo, descricao, quantidade, preco_unitario, total.
`vendas[].pagamentos[]`	lista	forma e valor; a soma é igual ao total (RN01).

4.3 Consumo demonstrado

O print abaixo é a execução real: login com curl, chamada à API e, por último, a mesma chamada sem sessão, que é recusada.

Terminal — consumo da API de integração com curl

```
$ curl -s -c sessao.txt -H "Content-Type: application/json" \
       -d '{"login":"rafael.gerente","senha":"Hermes@2026"}' http://localhost:3000/api/auth/login
{"usuario":{"id_usuario":2,"nome":"Rafael Lima","login":"rafael.gerente","perfil":"GERENTE","ativo":true,"criado_em":"2026-09-01 09:10"}}

$ curl -s -b sessao.txt "http://localhost:3000/api/integracao/vendas?inicio=2026-10-01&fim=2026-10-01" | python3 -m json.tool
{
    "sistema": "HermesPDV",
    "versao_layout": "1.0",
    "gerado_em": "2026-11-06T01:12:34.085Z",
    "periodo": {
        "inicio": "2026-10-01",
        "fim": "2026-10-01"
    },
    "quantidade": 3,
    "total": 130.46,
    "vendas": [
        {
            "numero": 1,
            "data_hora": "2026-10-01T09:41:17",
            "cliente": null,
            "atendente": "bianca.caixa",
            "subtotal": 59.13,
            "desconto": 0,
            "total": 59.13,
            "itens": [
                {
                    "codigo": "7891002902900",
                    "descricao": "Biscoito recheado morango 140g",
                    "quantidade": 2,
                    "preco_unitario": 3.59,
                    "total": 7.18
                },
                {
                    "codigo": "7891002202208",
                    "descricao": "Desinfetante lavanda 2L",
                    "quantidade": 2,
                    "preco_unitario": 10.99,
                    "total": 21.98
                },
                {
                    "codigo": "7891000700709",
                    "descricao": "Refrigerante cola 2L",
                    "quantidade": 3,
                    "preco_unitario": 9.99,
                    "total": 29.97
                }
            ],
            "pagamentos": [
                {
                    "forma": "PIX",
                    "valor": 59.13
    ... (demais vendas omitidas)

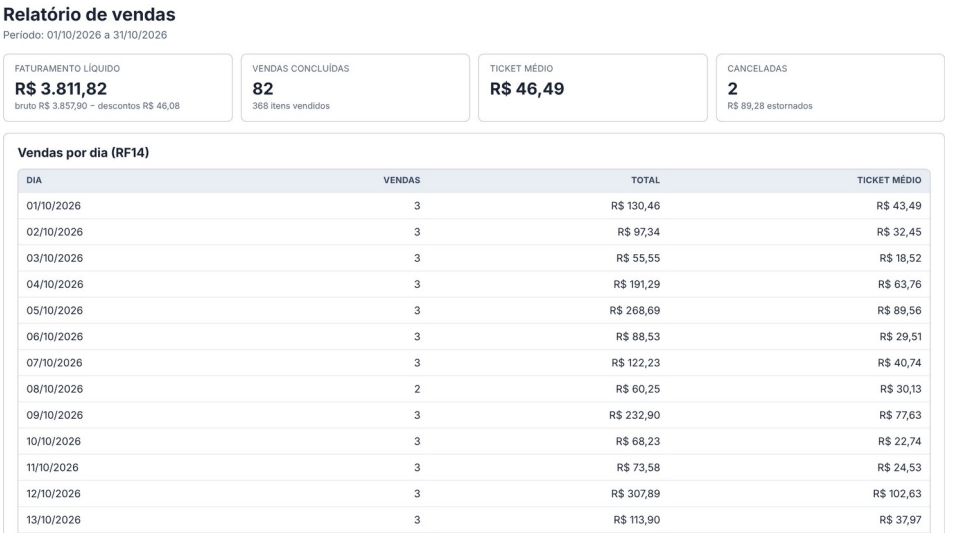
$ curl -s -i "http://localhost:3000/api/integracao/vendas"        # sem sessão
HTTP/1.1 401 Unauthorized
{"erro":"Sessão expirada ou inexistente. Entre novamente."}
```

4.4 Exportação do relatório

Além da API, o relatório de vendas exporta um CSV pensado para o Excel brasileiro (separador “;”, vírgula decimal e UTF-8 com BOM) e tem um layout de impressão próprio para salvar em PDF pelo navegador.



Relatório de vendas com totalizadores.



Layout de impressão (PDF): sem menu e sem botões.

5. Tabela de rastreabilidade

Todos os requisitos funcionais (RF01 a RF17), não funcionais (RNF01 a RNF06) e regras de negócio (RN01 a RN09) da Aula 2, com o status, a tela e o ponto do código onde estão.

Código	Requisito	Status	Tela	Onde no código
RF01	Cadastro de produtos	Implementado	Produtos (admin)	produtoService.js
RF02	Cadastro de clientes	Implementado	Clientes (admin)	clienteService.js
RF03	Cadastro de usuários com perfil	Implementado	Usuários (admin)	usuarioService.js
RF04	Autenticação por login e senha	Implementado	Login	authService.js, middlewares/auth.js
RF05	Registrar vendas com cliente e itens	Implementado	Nova venda (PDV)	vendaService.registrar
RF06	Calcular subtotal, desconto e total	Implementado	Nova venda (PDV)	pdv.js (tela) e vendaService (servidor)
RF07	Várias formas de pagamento	Implementado	PDV — pagamento	tabela forma_pagamento
RF08	Dividir o valor entre formas	Implementado	PDV — pagamento	vendaService.registrar
RF09	Baixa automática de estoque	Implementado	PDV (ao finalizar)	produtoRepository.movimentarEstoque
RF10	Alerta de estoque mínimo	Implementado	Painel, Produtos	alertaRepository.js
RF11	Consultar vendas por período, cliente ou produto	Implementado	Vendas	vendaRepository.listar
RF12	Cancelar venda repondo estoque	Implementado	Vendas	vendaService.cancelar
RF13	Comprovante em tela e impressão	Implementado	Comprovante	comprovante.js
RF14	Relatório de vendas por período	Implementado	Relatórios, Painel	relatorioRepository.porDia
RF15	Produtos mais vendidos	Implementado	Relatórios, Painel	relatorioRepository.maisVendidos
RF16	Total por forma de pagamento	Implementado	Relatórios, Painel	relatorioRepository.porFormaPagamento
RF17	Consultar estoque por nome ou código	Implementado	Produtos, PDV	produtoRepository.listar
RNF01	Responsivo (desktop e celular)	Implementado	Todas	estilo.css (@media)
RNF02	Senhas com hash	Implementado	Login, Usuários	crypt()/gen_salt('bf') — pgcrypto
RNF03	Registrar venda em até 2 s	Implementado	PDV	Medido: mediana de 14 ms em 30 vendas (servidor local)
RNF04	Integridade referencial	Implementado	—	9 chaves estrangeiras (Aula 5)
RNF05	Histórico após cancelamento	Implementado	Vendas, Comprovante	status CANCELADA; nada é apagado
RNF06	Venda em poucos cliques	Implementado	PDV	Código exato entra direto no carrinho; Enter busca
RN01	Pagamentos = total	Implementado	PDV — pagamento	vendaService.registrar
RN02	Sem venda acima do estoque	Implementado	PDV	vendaService + CHECK no banco
RN03	Cancelar repõe estoque	Implementado	Vendas	vendaService.cancelar
RN04	Perfis de acesso	Implementado	Menu e todas as telas	routes/index.js (autorizar)
RN05	Produto com venda só é inativado	Implementado	Produtos	produtoService.excluir
RN06	CPF/CNPJ único	Implementado	Clientes	clienteService + UNIQUE
RN07	Alerta no estoque mínimo	Implementado	PDV, Painel, Produtos	alertaRepository.gerarSeNecessario
RN08	Cancelamento exige motivo (sugerida na Aula 4)	Implementado	Vendas	vendaService.cancelar
RN09	Troco em dinheiro (sugerida na Aula 4)	Implementado	PDV — pagamento	vendaService.registrar

6. Qualidade do código e testes

- **Camadas:** rotas → controllers → services (regras) → repositories (SQL), como na arquitetura da Aula 4. Nenhum SQL fora dos repositórios e nenhuma regra nos controllers.
- **Segurança:** SQL sempre parametrizado; senha em bcrypt; cookie HttpOnly e SameSite; bloqueio após 5 tentativas; permissões checadas no servidor; o preço vem do banco; cabeçalhos de segurança.
- **Erros tratados:** um middleware único converte erros conhecidos em mensagens em português. Erros do banco (código duplicado, restrição violada, banco fora do ar) nunca chegam crus à tela, e o usuário não vê tela branca.
- **Consistência:** venda e cancelamento em transação única com bloqueio de linha; valores em centavos inteiros.
- **Testes:** 15 testes automáticos da API (npm test) cobrindo login, perfis, venda, cancelamento, RN01, RN02, RN05, RN06, RN07, os três CRUDs, relatório e API.

Terminal — testes automáticos da API (npm test)

```
$ npm test

▶ Autenticação e perfis (RF04, RN04)
  ✔ senha errada é recusada sem revelar qual campo errou (64.358596ms)
  ✔ usuário inativo não entra (61.98277ms)
  ✔ sem sessão não acessa a API (3.432679ms)
  ✔ atendente não cadastra produto e gerente não registra venda (13.276212ms)
✔ Autenticação e perfis (RF04, RN04) (326.26284ms)
▶ Venda e cancelamento (UC05, UC09)
  ✔ RN01: pagamentos diferentes do total são recusados (22.349957ms)
  ✔ RN02: não vende acima do estoque (12.99922ms)
  ✔ venda com pagamento dividido e troco; baixa e reposição de estoque (53.891241ms)
  ✔ RN07: venda que leva o produto ao mínimo gera alerta (57.188256ms)
✔ Venda e cancelamento (UC05, UC09) (147.175514ms)
▶ Cadastros (CRUD) com validação
  ✔ produto: criar, editar, excluir; código duplicado é recusado (39.346857ms)
  ✔ cliente: CPF inválido, CPF duplicado (RN06), criar, editar e excluir (21.345458ms)
  ✔ usuário: senha fraca recusada; criar, entrar, excluir (127.689227ms)
✔ Cadastros (CRUD) com validação (188.739621ms)
▶ Consultas, relatório e integração
  ✔ filtro de vendas por período e status (3.97589ms)
  ✔ relatório com totalizadores e exportação CSV (13.466876ms)
  ✔ dashboard traz os indicadores da Aula 3 (8.878628ms)
  ✔ API de integração devolve vendas com itens e pagamentos (3.237804ms)
✔ Consultas, relatório e integração (29.968063ms)
ℹ tests 15
ℹ suites 4
ℹ pass 15
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 777.44758
```

Execução dos testes automáticos: 15 de 15 aprovados.

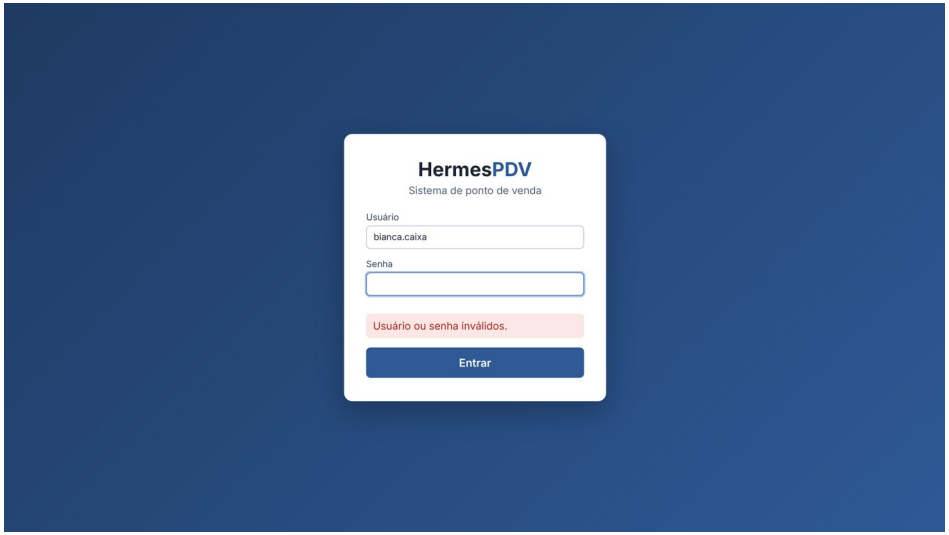
7. Trabalho em equipe

A aula exige commits reais de todos os integrantes, com branches e pull requests. Sugestão de divisão do código em quatro frentes, para que cada integrante seja responsável por uma branch, revise o código, faça os ajustes da sua parte e abra o pull request. Um colega revisa antes do merge.

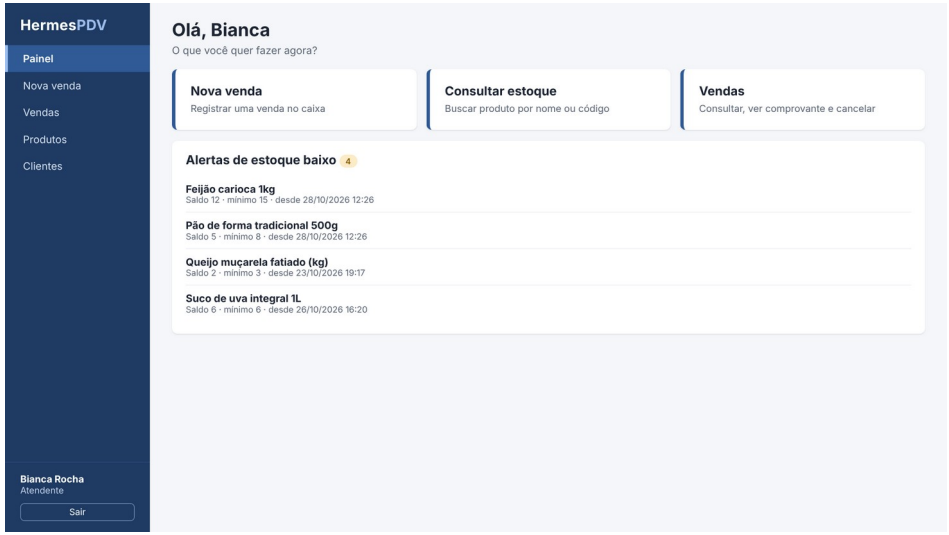
Branch	Frente	Arquivos principais	Integrante
feature/autenticacao	Login, sessão, perfis e usuários	backend: authService, usuarioService, middlewares/auth.js, utils/token.js · frontend: login, usuarios, app.js	
feature/cadastros	Produtos e clientes	backend: produtoService, clienteService, repositórios · frontend: produtos, clientes	
feature/venda	PDV, pagamento, comprovante e cancelamento	backend: vendaService, vendaRepository, alertaRepository · frontend: pdv, comprovante, vendas	
feature/gestao	Relatório, dashboard e API de integração	backend: relatorioService, relatorioRepository · frontend: painel, relatorios · testes	

- Commits pequenos e frequentes, com mensagem no imperativo dizendo o que mudou (ex.: “Adiciona filtro por produto na consulta de vendas”).
- Antes de cada pull request: npm test passando e o fluxo da frente testado na tela.
- Na apresentação final, cada integrante deve conseguir explicar a parte que assinou.

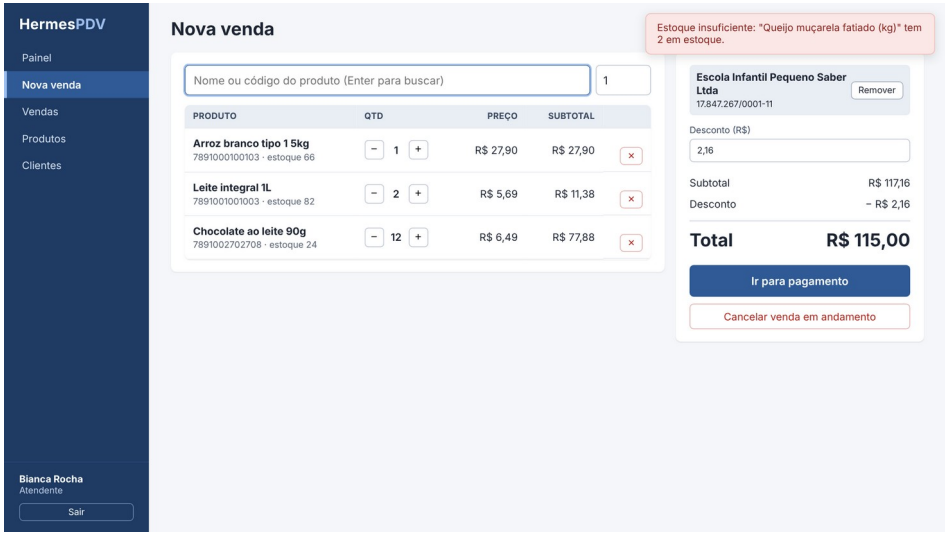
8. Telas do sistema



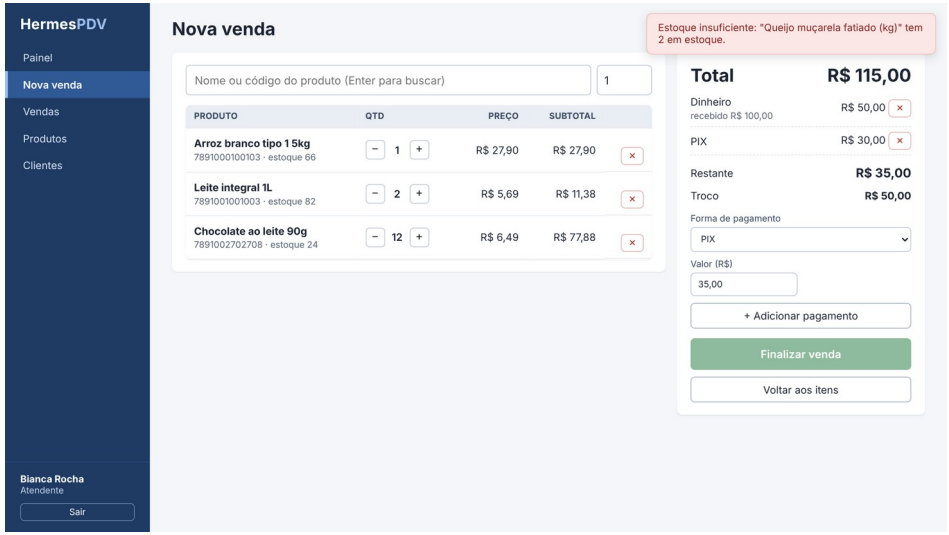
Login com senha errada: mensagem sem dizer qual campo errou.



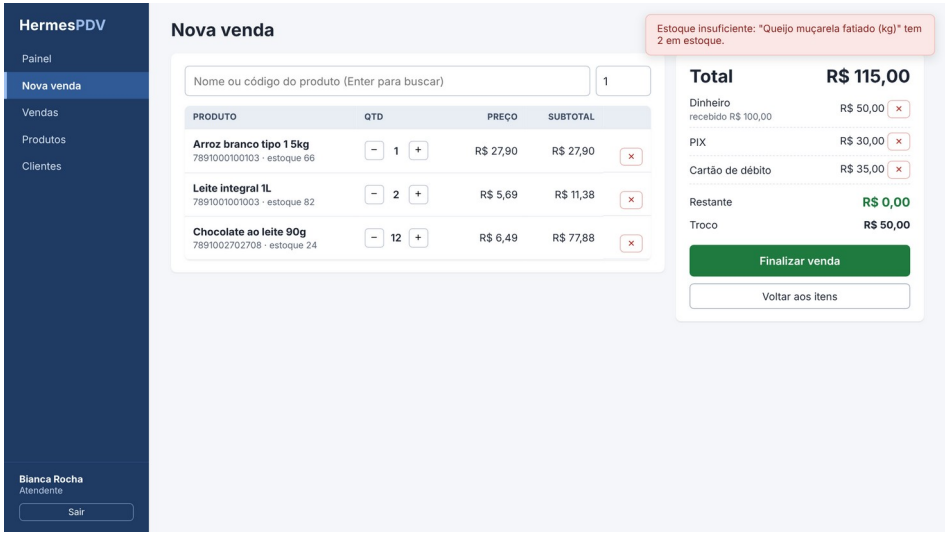
Painel da atendente: atalhos e alertas de estoque baixo.



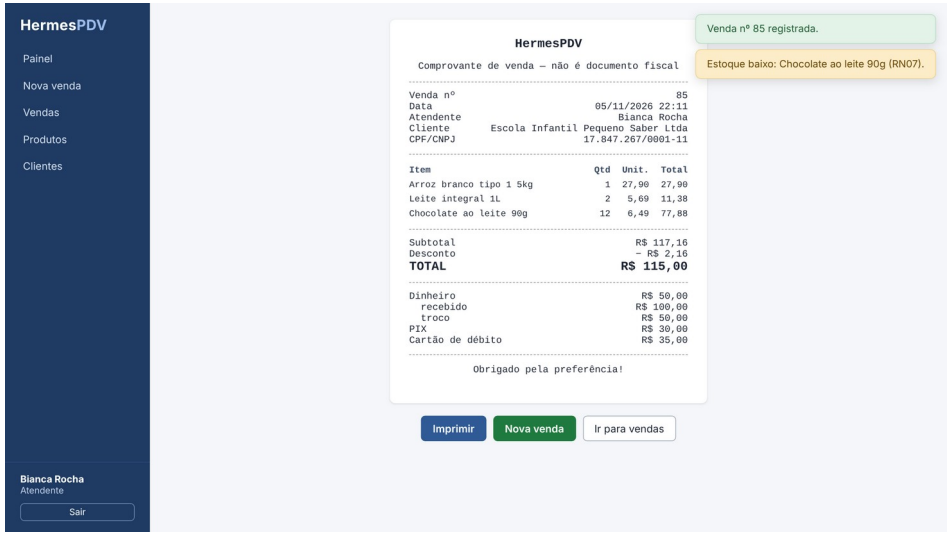
PDV: carrinho, cliente e desconto; RN02 bloqueia 5 kg de queijo com 2 em estoque.



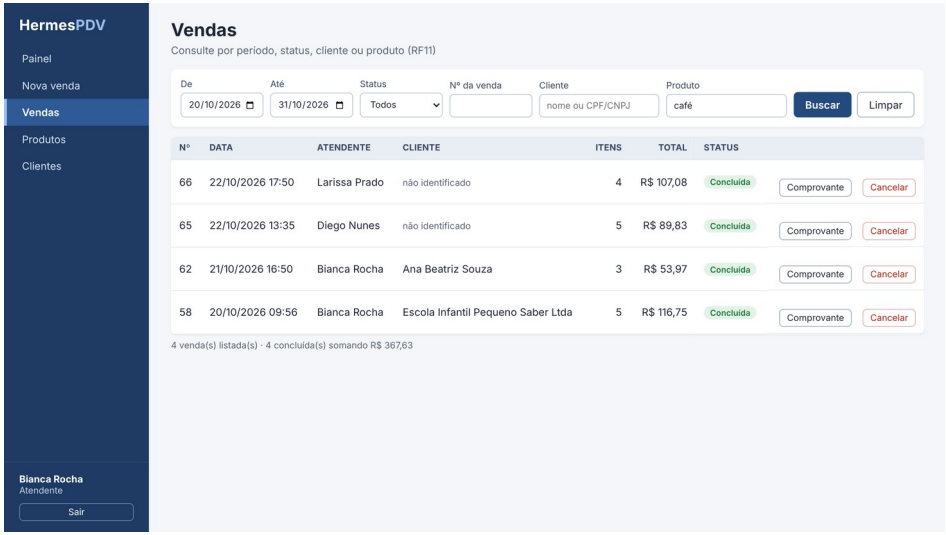
Pagamento dividido: dinheiro com troco e PIX; ainda falta pagar R$ 35,00.



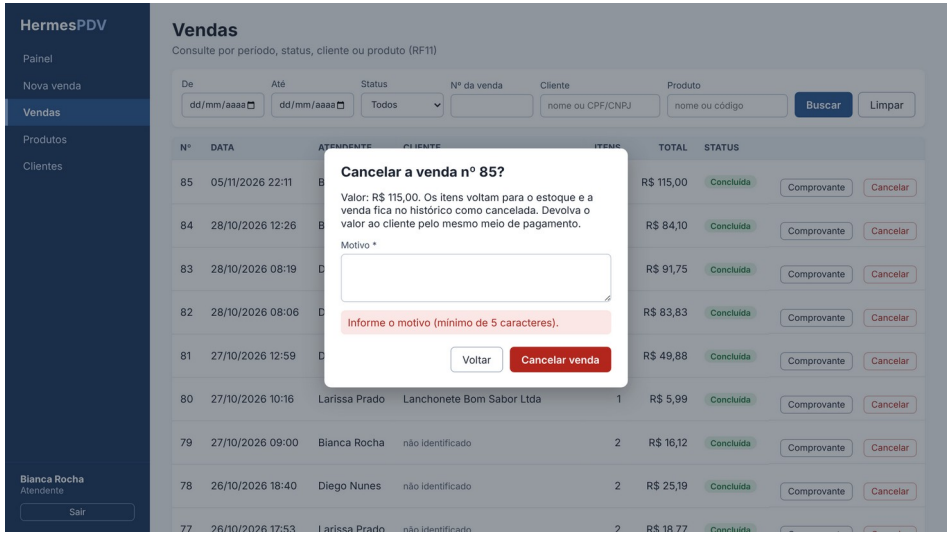
Restante zero: "Finalizar venda" liberado (RN01).



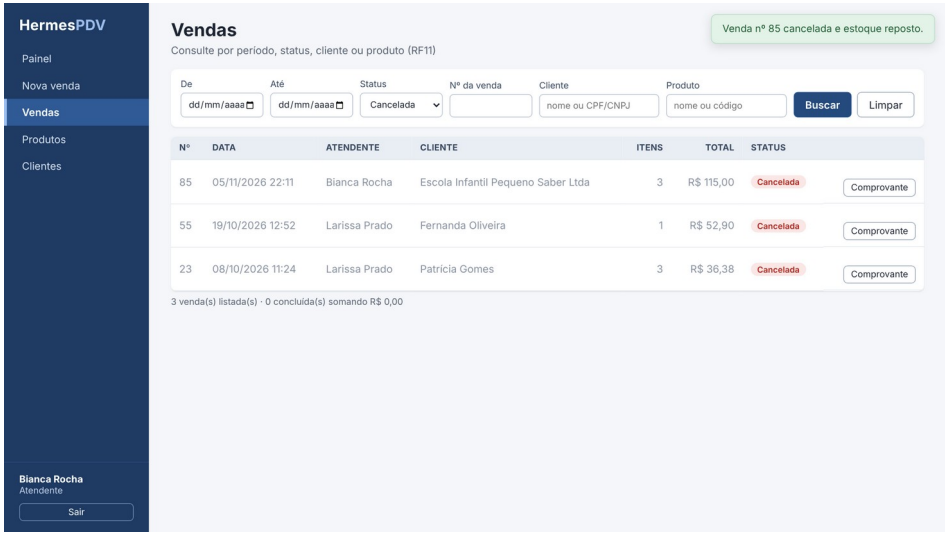
Comprovante, com avisos de venda registrada e de estoque baixo (RN07).



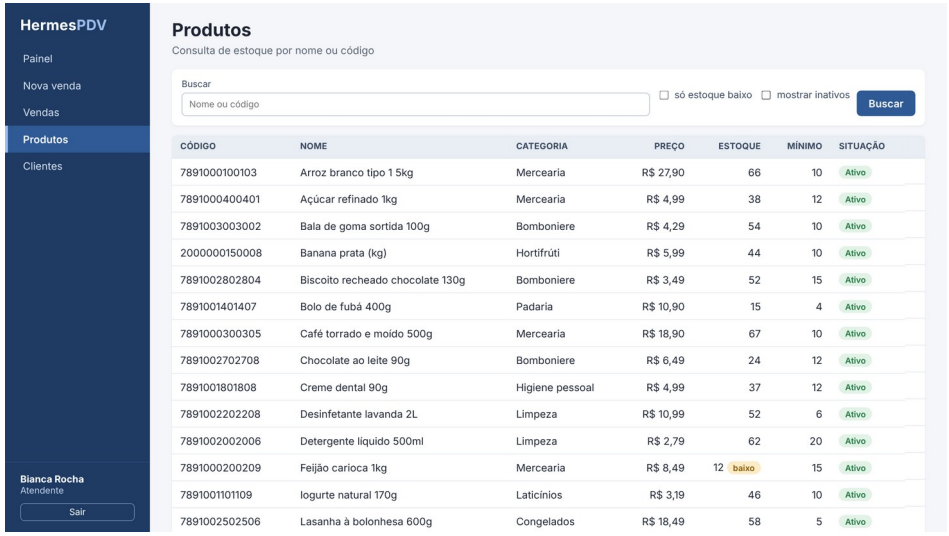
Consulta de vendas filtrada por período e produto (RF11).



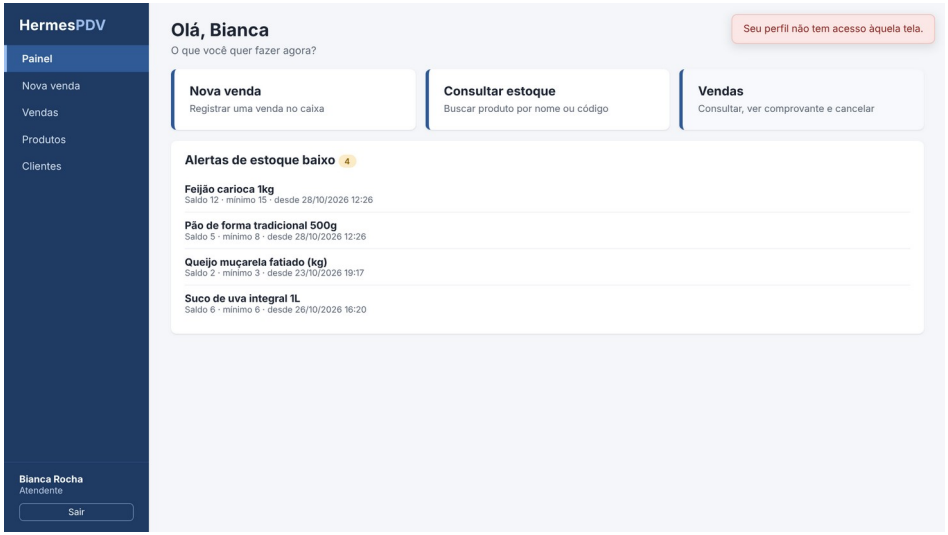
Cancelamento: o motivo é obrigatório (RN08).



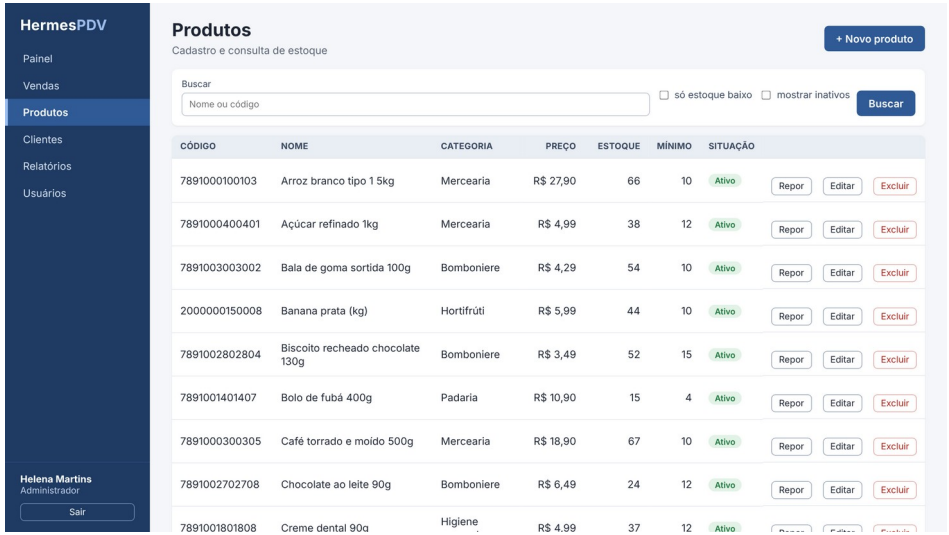
Venda cancelada no histórico; estoque reposto (RN03, RNF05).



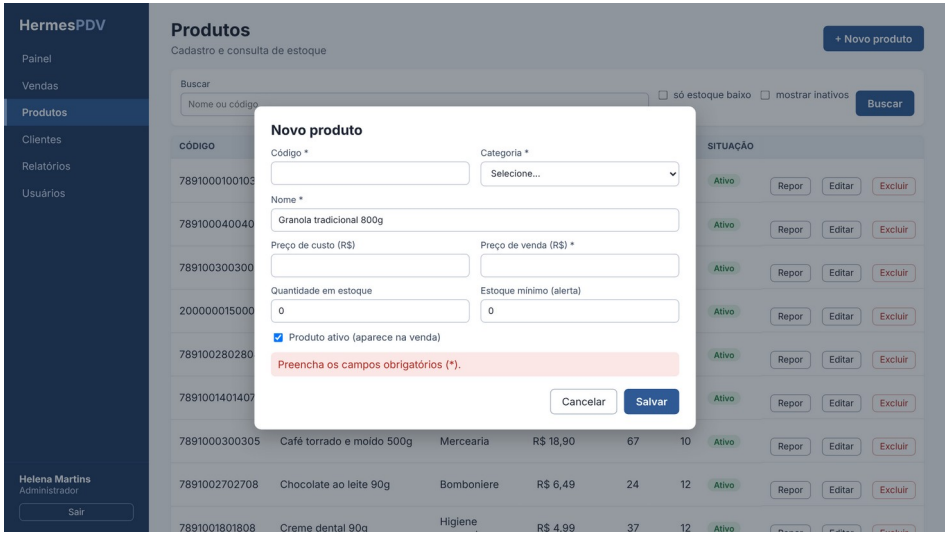
Atendente só consulta o estoque (RF17).



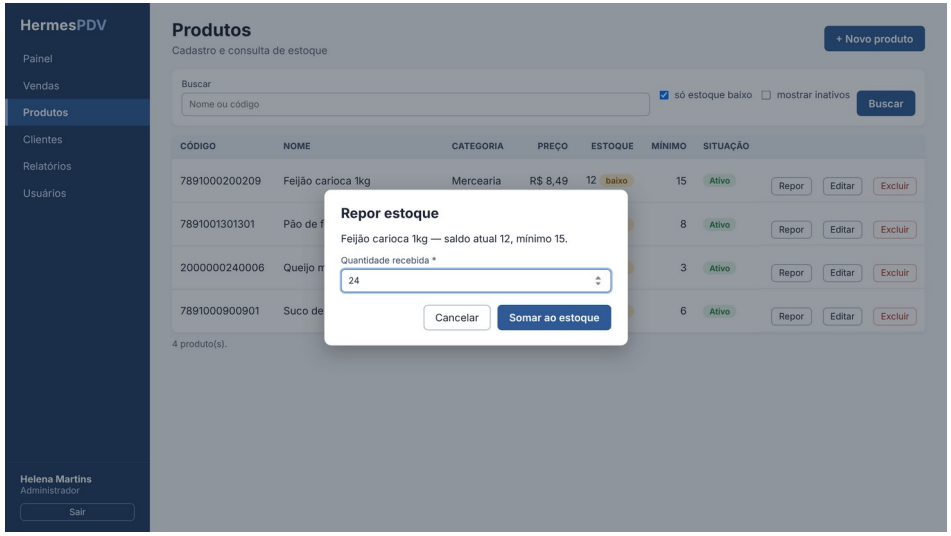
Tentativa de abrir uma tela proibida: volta ao painel com aviso.



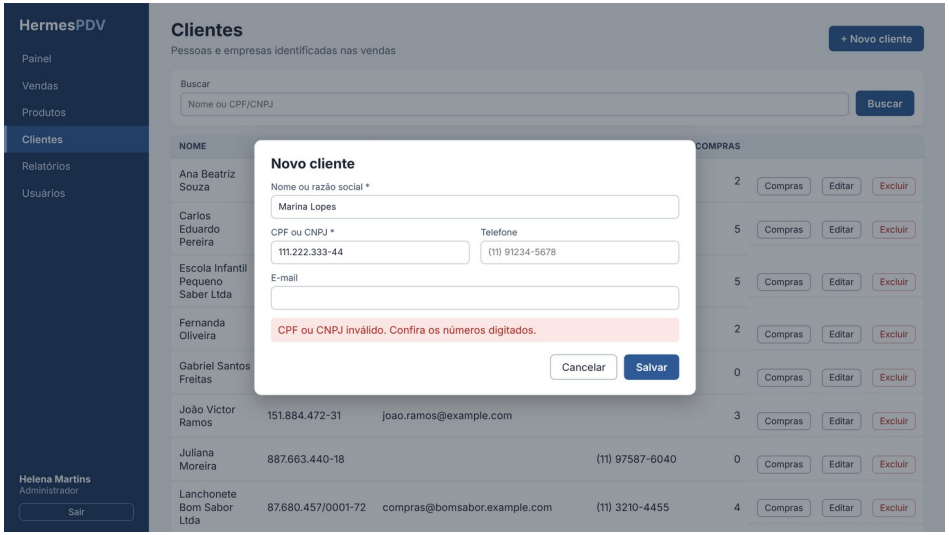
Produtos para o administrador: repor, editar e excluir.



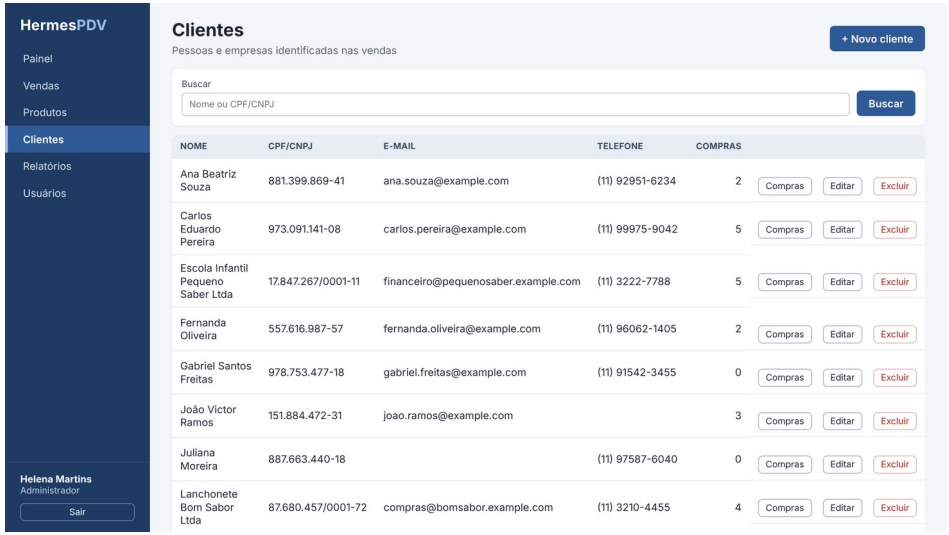
Validação de campos obrigatórios.



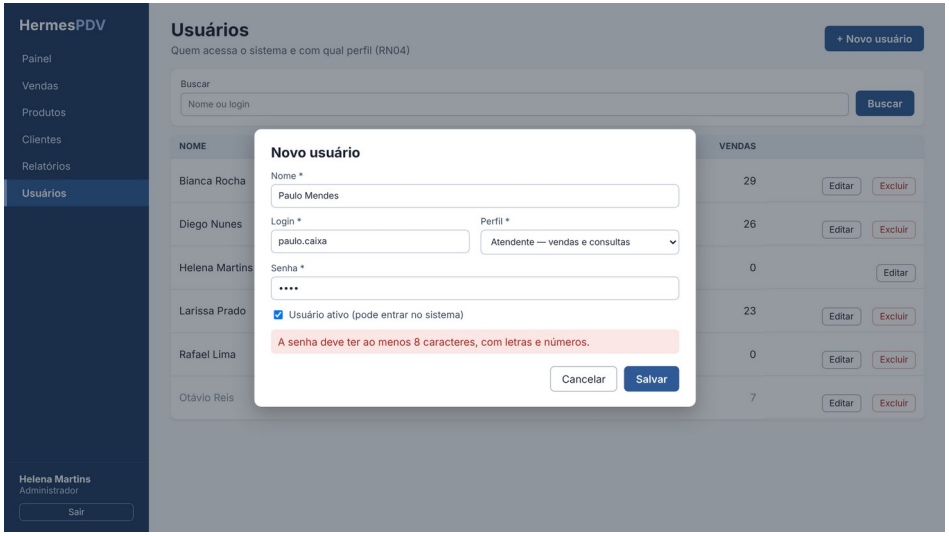
Reposição de estoque: encerra o alerta quando passa do mínimo.



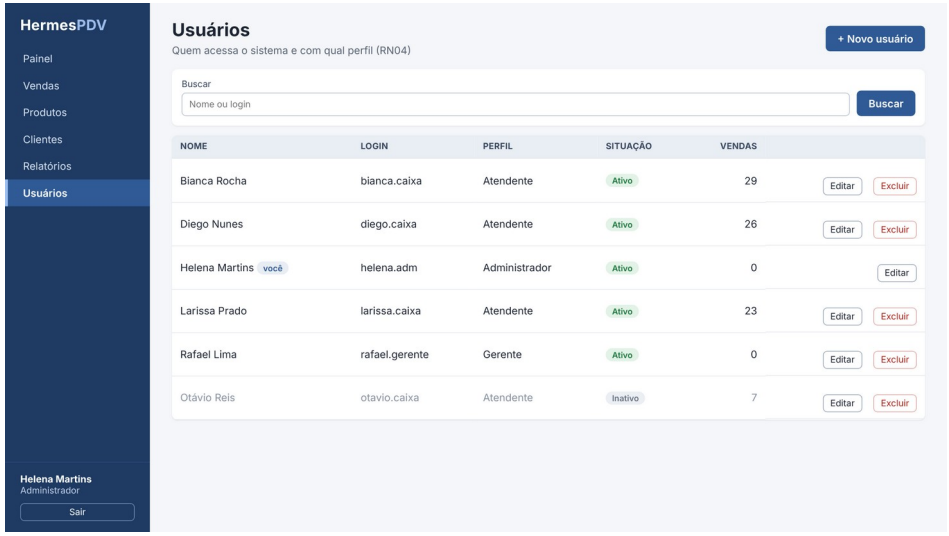
CPF com dígito verificador errado é recusado.



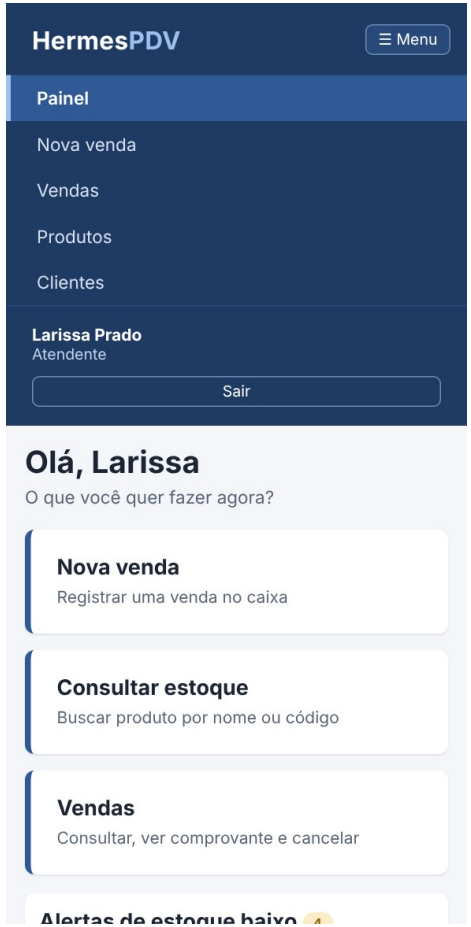
Cadastro de clientes.



Senha fraca recusada no cadastro de usuário.



Usuários: o administrador não pode excluir a si mesmo.



Celular: menu aberto.



Celular: venda.



Celular: consulta de vendas.

9. Repositório e aplicação

Link do repositório no GitHub:

Link da aplicação publicada (se houver):

O vídeo HermesPDV_Aula6_demonstracao_MVP.mp4 (7 min 21 s) acompanha esta entrega e mostra todos os fluxos com legendas: atendente, gerente, administrador e celular.